

AUTONOMOUS PRECISION LANDING ON A DYNAMIC PLATFORM

Using Vision-Based Tracking and Feedback Control

By: Lakshay Dhyani and Prithvi Gupta

1. Abstract

This project develops a simulation-based autonomous precision-landing system for a quadrotor landing on a moving rover in PyBullet. A downward-facing camera provides visual measurements using ArUco detection at higher altitude and template matching when the marker becomes partially visible at low altitude. A six-state Kalman filter estimates the rover's horizontal position, velocity, and acceleration.

The controller uses relative position and velocity errors with a SEARCH–TRACK–DESCEND–FINAL–CUTOFF state machine. Horizontal control combines rover acceleration feed-forward, position and velocity feedback, and disturbance compensation, while vertical control uses a scheduled descent reference. The resulting acceleration is converted into attitude, thrust, and four motor forces.

Recorded runs in both no-wind and wind-enabled arenas demonstrated successful rover tracking, controlled descent, touchdown, and motor cut-off. The reported numerical results correspond to these individual simulation runs rather than a statistically representative multi-trial evaluation.

2. Introduction

2.1 Background

Autonomous landing on a moving platform combines visual perception, motion estimation, trajectory tracking, attitude control, and touchdown constraints. Since the platform continuously changes position, the drone must estimate its position and motion while controlling its own movement.

2.2 Motivation

A purely vertical descent is unsuitable for a translating platform because relative position can increase as the rover moves. The controller therefore follows the rover during descent using its estimated position, velocity, and acceleration.

2.3 Problem Statement

The objective is to control a simulated quadrotor from approximately 3 m altitude to detect, track, and land on a moving rover. The drone must maintain suitable relative position and velocity, descend only when alignment conditions are satisfied, and safely reach the rover deck for motor cut-off.

2.4 Objectives

- Detect the moving rover using a downward-facing camera and ArUco marker.
- Maintain rover tracking when direct marker detection becomes unreliable.
- Estimate rover position, velocity, acceleration, and uncertainty.
- Track the rover through staged SEARCH, TRACK, DESCEND, FINAL, and CUTOFF modes.
- Generate acceleration commands and convert them into attitude, thrust, and four motor forces.
- Compensate for external disturbances in the wind-enabled arena.

2.5 Scope

The work is limited to the supplied PyBullet simulation and controller interface, including the moving rover, downward camera, ArUco marker, and optional wind disturbance. Real-UAV hardware, physical camera calibration, and flight-controller integration are outside the current scope.

3. System Overview

The complete control architecture is a closed loop in which simulated vehicle and camera measurements are converted into an estimated platform state, interpreted by a flight-state machine, and transformed into actuator forces. The

controller is called once per physics step. The core sequence is:

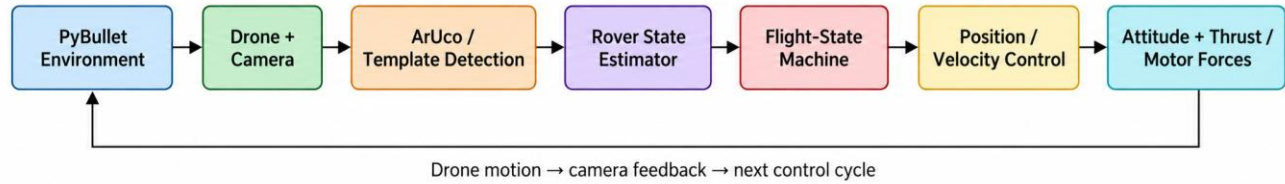


Figure 1. Overall system architecture of the implemented precision-landing controller.

- **PyBullet environment:** Provides vehicle dynamics, rendering, rover motion, wind disturbance, and actuator-force application.
- **Drone and camera:** A quadrotor with a 320×240 downward-facing camera.
- **Vision:** Detects the ArUco marker and provides template matching for low-altitude tracking.
- **Rover estimator:** Maintains horizontal position, velocity, acceleration, and uncertainty.
- **Flight controller:** Manages flight modes, references, acceleration commands, attitude, thrust, and motor mixing.
- **Wind estimator:** Estimates external disturbance force from the drone's motion and applied thrust.
- **Safety logic:** Handles touchdown and motor cut-off using the arena's landing constraints.

4. Simulation Environment

Two supplied PyBullet arenas were used: a **standard no-wind arena** and a **wind-enabled arena**. Both operate at a simulation timestep of $1/240$ s and use the same controller interface. The wind-enabled arena additionally applies a time-varying external force to the drone.

Parameter	Implemented value	Significance
Physics timestep	$1/240 \text{ s} = 0.004167 \text{ s}$	Sets the controller and physics update interval.
Gravity	9.81 m/s^2 downward	Defines the gravitational acceleration used by PyBullet and the controller.
Drone mass	1.20 kg	1.0 kg chassis + four 0.05 kg rotor links.
Motor arm	0.25 m	Distance of each motor from the body center along x/y.
Initial drone position	$[0, 0, 3.0] \text{ m}$	Initial altitude for the local test.
Rover chassis	$1 \text{ m} \times 1 \text{ m} \times 0.1 \text{ m}$	Defines the landing surface and white margin.
Marker	0.8 m wide	Landing target rendered on the rover.
Camera	$320 \times 240 \text{ px}$	Image resolution supplied to the controller.
Vertical FOV	60°	Used by the PyBullet projection model.
Camera offset	0.10 m below drone center	Determines camera position.
Deck height	0.102 m	Approximate marker/deck surface height.
Touch height	0.14 m	Drone reference height for deck contact.
Cruise height	3.0 m	Nominal height used while tracking.
Search height	3.6 m	Reference height used during SEARCH.
Vision floor	$0.102 + 0.10 + 0.18 = 0.382 \text{ m}$	Below this, direct camera observation is treated as unreliable.
Cut height	0.152 m	CUTOFF threshold derived from touch height + 0.012 m.
Controller rate	240 Hz	One control call per physics step.

In the wind-enabled arena, horizontal disturbances range from **-2 to +2 N** and vertical disturbances from **-0.5 to +0.5 N**. The target wind is updated every 120 simulation steps and smoothly interpolated to produce changing external forces.

5. Drone Model and Dynamics

The simulated vehicle is a four-motor quadrotor. The base chassis has a mass of 1.0 kg and each of the four rotor links has a mass of 0.05 kg, giving a controller mass of 1.20 kg. The motors are positioned at $(+L, +L)$, $(-L, +L)$, $(+L, -L)$, and $(-L, -L)$, where $L = 0.25$ m.

Total vehicle mass: $m = 1.0 + 4(0.05) = 1.20$ kg

The four upward motor forces produce the total thrust magnitude.

$$\mathbf{F}_T = \mathbf{f}_1 + \mathbf{f}_2 + \mathbf{f}_3 + \mathbf{f}_4$$

The front/back force imbalance creates roll torque about the body x-axis.

$$\tau_{\text{roll}} = L * (\mathbf{f}_1 + \mathbf{f}_2 - \mathbf{f}_3 - \mathbf{f}_4)$$

The left/right force imbalance creates pitch torque about the body y-axis.

$$\tau_{\text{pitch}} = L * (-\mathbf{f}_1 + \mathbf{f}_2 - \mathbf{f}_3 + \mathbf{f}_4)$$

The arena applies vertical actuator forces directly at the four motor locations. Therefore, the controller outputs **four motor forces rather than motor speeds**, and independent yaw torque is not commanded by the supplied actuator model.

6. Camera and Coordinate Transformation

The camera is rigidly associated with the drone and is located 0.10 m below the drone center along the body z direction. The simulated camera looks downward along body $-z$, with body $+x$ defining the image-up direction and body $-y$ corresponding to the image-right direction. The controller represents the vehicle attitude by rotation matrix R that maps body-frame vectors into the world frame.

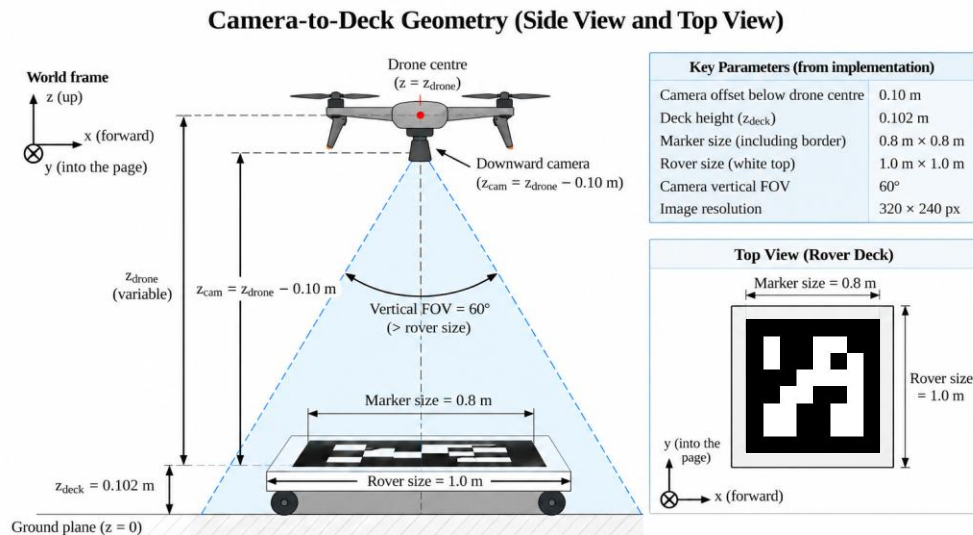


Figure 2. Camera-to-deck geometry showing the side view and top view of the rover. The camera is located 0.10 m below the drone centre and looks downward with a vertical field of view of 60°, which is wider than the rover size. The rover deck is 1.0 m $\times$ 1.0 m and the ArUco marker occupies 0.8 m $\times$ 0.8 m of the deck surface.

A pixel is converted into a camera/body ray using the implemented focal length and image center.

$$\mathbf{r}_{\text{body}} = \begin{bmatrix} -(v - C_y)/f \\ -(u - C_x)/f \\ -1 \end{bmatrix}$$

$\mathbf{r}_{\text{body}}$ = ray expressed in drone/body coordinated

u = pixel x-coordinate

v = pixel y-coordinate

C_x, C_y = image center

f = camera focal length

$$f = \frac{H/2}{\tan(FOV/2)}$$

H = vertical resolution of camera image = 240 px

FOV = Field of View = 60°

$f \approx 208\text{px}$

The body-frame viewing ray is rotated into the world frame.

$$\mathbf{r}_{\text{world}} = \mathbf{R} * \mathbf{r}_{\text{body}}$$

$\mathbf{r}_{\text{world}}$ = ray expressed in world coordinates

$\mathbf{R}$ = drone rotation matrix

The ray is intersected with the horizontal deck plane.

$$s = (z_{\text{deck}} - z_{\text{cam}}) / \mathbf{r}_{\text{world},z}$$

z_{deck} = Deck height = 0.102m

z_{cam} = Vertical world coordinate height of camera

$\mathbf{r}_{\text{world},z}$ = z component of camera ray in world frame

s = distance/scaling factor along the camera ray needed to reach the deck

The resulting point supplies the world x-y location associated with the image pixel.

$$\mathbf{p}_{\text{deck}} = \mathbf{p}_{\text{cam}} + s * \mathbf{r}_{\text{world}}$$

$\mathbf{p}_{\text{cam}}$ = 3D position of camera in world frame consisting of $x_{\text{cam}}, y_{\text{cam}}$ and z_{cam}

$\mathbf{p}_{\text{deck}}$ = 3D point where the camera ray intersects the rover deck

The reverse mapping is implemented by a homography-like deck-to-image transformation. It projects a world deck point into image coordinates and is used by the template matcher and debug overlay.

7. Vision-Based Platform Detection

The platform is detected using a two-stage vision pipeline. At higher altitude, the full ArUco marker is generally visible and is used for direct detection. As the drone approaches the platform, the marker can become partially visible, so the controller switches to template matching.

7.1 ArUco Marker Detection

Multiple OpenCV ArUco dictionaries are checked to identify the required marker. Once the target marker ID is locked, its image position is converted into a deck position using the camera transformation described in Section 6.

7.2 High-Altitude Detection

At higher altitude, the complete marker is usually visible. ArUco detection provides the primary visual measurement because the marker can be identified directly and reliably.

7.3 Low-Altitude Tracking Problem

Near the platform, perspective and the camera's limited field of view can cause only part of the marker to remain visible. Full-marker ArUco detection can therefore become unavailable even though useful visual information is still present.

7.4 Template Matching

Once a valid ArUco observation has established the marker pattern and orientation, the controller constructs a synthetic grayscale view containing the rover floor, white pad, and marker pattern. For subsequent low-altitude frames, the expected view is warped through the current camera and deck geometry and compared with the real image using normalized template correlation. The search radius is tied to the rover-position uncertainty and is bounded to prevent an excessively broad match.

$$r_{\text{search}} = \text{clip}(3\sigma_{\text{position}}, r_{\text{min}}, r_{\text{max}})$$

r_{search} = radius of the area in which template matching searches for the marker

σ_{position} = uncertainty in the estimated rover/marker position

r_{min} = minimum search radius = 0.025m

r_{max} = maximum search radius

$\text{clip}(x, \text{minimum}, \text{maximum})$ means keep x between minimum and maximum values

The search area expands with state uncertainty, with a maximum radius of 0.05 m below 0.8 m camera distance and 0.20 m otherwise.

A match is accepted only when its normalized score is at least 0.55, the best peak is not on the search edge, and the measurement passes the rover tracker surprise test. The correlation peak is refined locally and its curvature is used to construct a measurement covariance. This is the implemented reason for combining direct marker detection with model-based template matching.

8. Platform State Estimation

8.1. Kalman Filter Model

The detected rover position is processed by a six-state Kalman filter to obtain a smoother estimate of rover motion.

Kalman State	Prediction and Matrix Form	Terminology
$\mathbf{x}_k = \begin{bmatrix} x_k \\ y_k \\ v_{x,k} \\ v_{y,k} \\ a_{x,k} \\ a_{y,k} \end{bmatrix}$	Prediction equations	x_k = rover's x-position at time k
	$x_{k+1}^- = x_k + v_{x,k}\Delta t + \frac{1}{2}a_{x,k}\Delta t^2$	y_k = rover's y-position at time k
	$y_{k+1}^- = y_k + v_{y,k}\Delta t + \frac{1}{2}a_{y,k}\Delta t^2$	$v_{x,k}$ = rover's x-velocity
	$v_{x,k+1}^- = v_{x,k} + a_{x,k}\Delta t$	$v_{y,k}$ = rover's y-velocity
	$v_{y,k+1}^- = v_{y,k} + a_{y,k}\Delta t$	$a_{x,k}$ = rover's x-acceleration
	$a_{x,k+1}^- = a_{x,k}$	$a_{y,k}$ = rover's y-acceleration
	$a_{y,k+1}^- = a_{y,k}$	Δt = time between updates
	Matrix form	x_{k+1}^- = predicted state at $k + 1$
	$\mathbf{x}_{k+1}^- = F\mathbf{x}_k$	P_k = state covariance
	$P_{k+1}^- = FP_kF^T + Q$	F = state-transition matrix
		Q = process-noise covariance

8.2. Measurement Correction

When a valid camera measurement is available, the predicted rover state is corrected using the measurement:

Equation	Terminology
$\mathbf{r}_{k+1} = \mathbf{z}_{k+1} - H\mathbf{x}_{k+1}^-$	$\mathbf{z}_{k+1}$ = new camera measurement
$S_{k+1} = HP_{k+1}^-H^T + R$	$\mathbf{x}_{k+1}^-$ = predicted rover state
$K_{k+1} = P_{k+1}^-H^TS_{k+1}^{-1}$	H = measurement matrix
$\mathbf{x}_{k+1} = \mathbf{x}_{k+1}^- + K_{k+1}\mathbf{r}_{k+1}$	$\mathbf{r}_{k+1}$ = measurement residual
$P_{k+1} = (I - K_{k+1}H)P_{k+1}^-$	S_{k+1} = innovation covariance
	K_{k+1} = Kalman gain
	P_{k+1}^- = predicted covariance
	P_{k+1} = corrected covariance
	R = measurement-noise covariance
	I = identity matrix

8.2. Mahalanobis Validation

Before applying the correction, the camera measurement is checked against the predicted state:

$$d_M^2 = \mathbf{r}_{k+1}^T S_{k+1}^{-1} \mathbf{r}_{k+1}$$

The squared Mahalanobis distance measures the difference between the camera measurement and the predicted rover state while accounting for their uncertainty. Measurements exceeding the specified threshold are rejected before Kalman correction

The implementation forms the corresponding 6×6 transition and process-noise matrices for x and y. The initial covariance assigns 0.05 m standard deviation to position and 1.5 m/s and 1.5 m/s² to velocity and acceleration. Position-only measurements are incorporated through a standard Kalman correction. The measurement innovation is tested using a squared Mahalanobis distance before correction.

9. Relative Motion Estimation

The controller uses the estimated rover state and the drone state to calculate motion relative to the landing platform. This allows the controller to determine whether the drone is sufficiently aligned and moving slowly enough to begin descent.

Position error: $\mathbf{e}_p = \mathbf{p}_{\text{rover}} - \mathbf{p}_{\text{drone}}$

Velocity error: $\mathbf{e}_v = \mathbf{v}_{\text{rover}} - \mathbf{v}_{\text{drone}}$

Relative distance: $d = ||\mathbf{e}_p||$

Relative speed: $\Delta v = ||\mathbf{e}_v||$

The relative distance and relative speed are used by the flight-state machine to determine whether the drone is sufficiently aligned for descent. The rover estimate can remain available through prediction during temporary visual loss, with the implementation allowing an estimate to be used for up to approximately **2.5 s of staleness**.

10. Wind and External Disturbance Handling

The controller does not assume that the wind-enabled arena is known in advance. Instead, it estimates external force from the measured change in drone velocity and the thrust vector that the controller applied during the previous step.

$$\mathbf{F}_{\text{ext}} = m \frac{\mathbf{v}_k - \mathbf{v}_{k-1}}{\Delta t} - \mathbf{F}_{\text{thrust}} - \mathbf{F}_{\text{gravity}}$$

The measured net force is reduced by the controller's own thrust and gravity; the residual is interpreted as external force.

Near the deck, the estimator is disabled because contact forces from the rover should not be interpreted as wind. Above the touchdown region, the raw disturbance sample is clipped to ± 6 N per axis and filtered with an approximately 50 ms time constant. The horizontal magnitude is then averaged over approximately 1 s for wind-state detection.

Wind element	Implemented value / behavior
Wind ON threshold	0.30 N average horizontal force
Wind OFF threshold	0.12 N
Sample clipping	± 6 N per axis
Extra tilt authority	+8° when wind is detected

The disturbance estimate is filtered with a **50 ms filter** and averaged over approximately **1 s**. Wind compensation is disabled near the landing deck to avoid unnecessary corrections during touchdown.

11. Flight-State Machine

The controller uses five explicit phases: SEARCH, TRACK, DESCEND, FINAL, and CUTOFF.

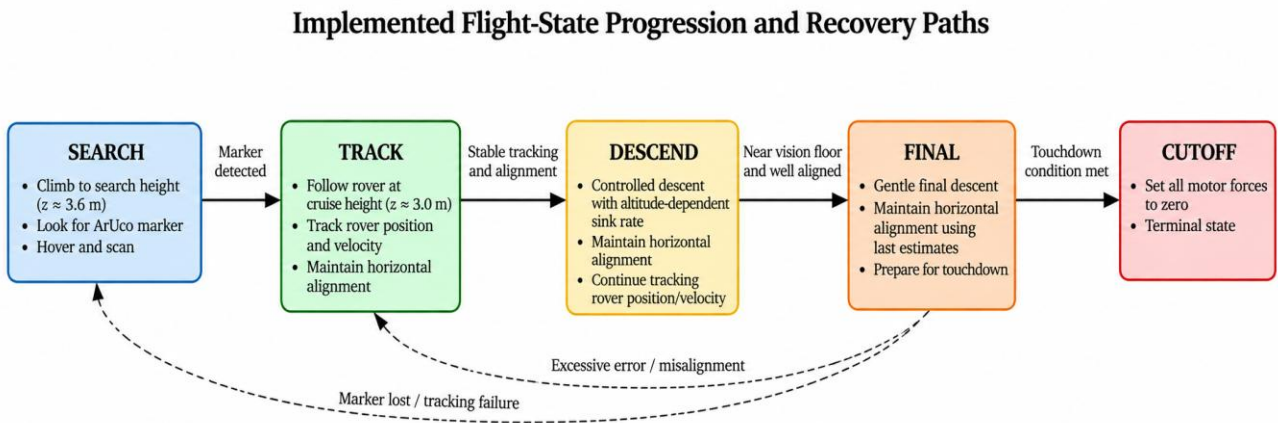


Figure 3. Implemented flight-state progression and principal recovery paths.

State	Functions	Transition condition
SEARCH	Locate and acquire the rover marker	Valid rover detection

TRACK	Follow the rover while maintaining altitude.	Stable alignment and low relative motion
DESCEND	Reduce altitude while continuing horizontal tracking.	Conditions remain valid
FINAL	Perform controlled low-altitude descent with tighter position control.	Near landing height
CUTOFF	Stop motor forces after touchdown conditions are satisfied.	Landing constraints met

During TRACK, the descent qualification condition is horizontal distance below 0.12 m, relative speed below 0.25 m/s, and visual staleness below 0.1 s, maintained for more than 0.6 s. During DESCEND, excessive position error, relative speed, or visual staleness suppresses descent. Near the vision floor, the controller either commits to FINAL when alignment remains adequate or climbs/retries when it is not.

12. Horizontal Position Control

The horizontal controller tracks the moving rover using relative position and velocity errors while incorporating the rover's estimated acceleration and external disturbance compensation.

$$\mathbf{a}_{xy} = \mathbf{a}_{\text{rover}} + K_p \mathbf{e}_p + K_d \mathbf{e}_v - \frac{\mathbf{F}_{\text{wind},xy}}{m}$$

Condition	K_p	K_d	Maximum Tilt
Normal tracking	4.0	4.2	22°
FINAL / below 1 m	7.0	5.0	15°

$\mathbf{a}_{xy}$ = desired horizontal acceleration
 $\mathbf{a}_{\text{rover}}$ = rover acceleration
 $\mathbf{e}_p$ = position error
 $\mathbf{e}_v$ = velocity error
 K_p = position gain
 K_d = velocity gain
 $\mathbf{F}_{\text{wind},xy}$ = estimated horizontal wind force
 $\mathbf{a}_{\text{max}}$ = maximum horizontal acceleration
 θ_{max} = maximum allowed drone tilt angle

If wind is detected, 8° is added to the maximum tilt allowance.

The allowable horizontal acceleration is limited by the maximum commanded tilt angle.

$$\mathbf{a}_{\text{max}} = g \tan(\theta_{\text{max}})$$

13. Vertical Control

The vertical controller generates the required vertical acceleration using the altitude and vertical-velocity errors relative to a scheduled descent reference.

$$\mathbf{a}_z = 16(\mathbf{z}_{\text{ref}} - \mathbf{z}) + 8(\mathbf{v}_{z,\text{ref}} - \mathbf{v}_z) - \mathbf{F}_{\text{wind},z} / m$$

The vertical controller uses proportional altitude correction, derivative velocity correction, and vertical disturbance compensation.

$\mathbf{a}_z$ = desired vertical acceleration
 $\mathbf{z}_{\text{ref}}$ = reference altitude the controller wants the drone to move toward
 $\mathbf{z}$ = current altitude
 $16(\mathbf{z}_{\text{ref}} - \mathbf{z})$ = altitude correction
 $\mathbf{v}_{z,\text{ref}}$ = reference vertical velocity
 $\mathbf{v}_z$ = actual vertical velocity
 $8(\mathbf{v}_{z,\text{ref}} - \mathbf{v}_z)$ = velocity correction
 $\mathbf{F}_{\text{wind},z}$ = vertical wind force

The altitude reference is smoothly reduced according to the current landing state. The controller uses:

- **Search altitude:** 3.6 m
- **Cruise altitude:** 3.0 m

- **Final descent velocity:** approximately -0.35 m/s
- **Reference-velocity slew limit:** 1.2 m/s^2

The commanded vertical acceleration is limited to:

$$-6 \leq a_z \leq 8 \text{ m/s}^2$$

The reference velocity is also rate-limited to prevent abrupt changes in the descent command.

14. Acceleration to Attitude and Thrust

The controller converts the desired world acceleration into a force vector that includes gravity, then constructs an attitude whose body-up axis points along the desired force while preserving the current yaw heading.

$$\mathbf{F}_{des} = m * (\mathbf{a}_{des} + \begin{bmatrix} 0 \\ 0 \\ g \end{bmatrix}) ; \text{ where } \mathbf{a}_{des} = \begin{bmatrix} a_{xy,x} \\ a_{xy,y} \\ a_z \end{bmatrix}$$

The desired physical force must both accelerate the drone and counteract gravity.

The normalized desired force defines the required thrust direction:

$$\mathbf{u}_3 = \frac{\mathbf{F}_{des}}{\|\mathbf{F}_{des}\|}$$

$$\mathbf{u}_2 = \frac{\mathbf{u}_3 * \mathbf{h}}{\|\mathbf{u}_3 * \mathbf{h}\|}$$

$$\mathbf{u}_1 = \mathbf{u}_2 * \mathbf{u}_3$$

$$\text{where } \mathbf{h} = \begin{bmatrix} \cos \psi \\ \sin \psi \\ 0 \end{bmatrix} \text{ and } \psi = \text{atan2}(R_{12}, R_{11})$$

a_{des} = desired acceleration
 g = acceleration due to gravity
 F_{des} = desired force
 $\mathbf{u}_3$ = unit vector pointing in the required thrust direction
 R = current rotation matrix/orientation of the drone
 R_{des} = desired rotation matrix
 $\mathbf{u}_1$ = desired forward direction
 $\mathbf{u}_2$ = desired left direction
 $\mathbf{u}_3$ = desired up/thrust direction
 $\mathbf{e}_R$ = Difference between current and desired orientation
 $\text{vee}()$ = converts a skew-symmetric matrix into a 3D vector
 τ = torque
 I = moment of inertia = 0.018
 ω = actual angular velocity
 ω_n = attitude-controller natural frequency = 14 rad/s
 ζ = damping ratio = 0.9

A horizontal heading vector is formed from the current yaw. Cross products generate the desired left and forward body axes, producing the desired rotation matrix R_{des} . The attitude error is obtained from the skew-symmetric part of the relative rotation.

$$\mathbf{R}_{des} = [\mathbf{u}_1 \quad \mathbf{u}_2 \quad \mathbf{u}_3]$$

$$\mathbf{e}_R = \frac{1}{2} * \text{vee}(\mathbf{R}_{des}^T \mathbf{R} - \mathbf{R}^T \mathbf{R}_{des})$$

$$\tau = -I * \omega_n^2 * \mathbf{e}_R - I * (2 * \zeta * \omega_n) * \omega$$

Roll and pitch torque are limited to ± 1.2 in the controller's force/torque units. Total thrust is the component of the desired force along the current body-up axis, clipped at zero before motor allocation.

15. Motor Force Allocation

The final motor forces are produced by combining equal base thrust with roll and pitch contributions. With $ARM = 0.25 \text{ m}$:

$$\text{base} = F_T/4; \text{roll} = \tau_x/(4L); \text{pitch} = \tau_y/(4L)$$

Base thrust shares the total load equally; torque terms create the required force differences.

$$f_1 = \text{base} + \text{roll} - \text{pitch}$$

$$f_2 = \text{base} + \text{roll} + \text{pitch}$$

$$f_3 = \text{base} - \text{roll} - \text{pitch}$$

$$f_4 = \text{base} - \text{roll} + \text{pitch}$$

Motor positions: (+L, +L), (-L, +L), (+L, -L), (-L, -L)

Each final force is clamped to a minimum of zero because the simulated motors can push but cannot pull. The mixing equations are consistent with the torque relationships defined for the four actuator locations.

16. Complete Controller Algorithm

The engineering sequence executed by `calculate_forces()` is:

1. **Read simulator inputs:** At each 240 Hz control cycle, read the camera frame, drone position, orientation, and velocity from the simulator.
2. **Check whether CUTOFF has already occurred:** If the controller is already in the CUTOFF state, set all four motor forces to zero and terminate the current control cycle. Otherwise, continue with the control pipeline.
3. **Estimate angular velocity:** Compute the drone's angular velocity from the change in its rotation matrix between the current and previous control cycles.
4. **Estimate and detect wind:** Estimate the external disturbance force from the drone's motion and previously applied thrust. Filter the estimate, determine whether wind is present using the ON/OFF thresholds, and increase the available tilt authority when wind is detected.
5. **Perform Kalman prediction:** Predict the rover's horizontal position, velocity, and acceleration using the six-state constant-acceleration Kalman model.
6. **Construct camera geometry:** Generate the current downward-facing camera model using the drone pose, camera offset, field of view, and image dimensions.
7. **Process the camera image:** Detect the rover marker using ArUco detection when the marker is sufficiently visible. At low altitude or when the marker becomes partially visible, use the previously generated synthetic template and search around the predicted rover location.
8. **Perform measurement validation:** Compare the detected measurement with the predicted rover state using the Mahalanobis-distance test. Accept the measurement only when it satisfies the corresponding threshold and validation conditions.
9. **If accepted:** Perform the Kalman correction using the new measurement.
10. **If rejected:** Do not update the Kalman filter with the measurement and continue using the predicted rover state.
11. **Obtain the rover estimate:** Extract the rover's estimated position, velocity, and acceleration from the corrected or predicted Kalman state, provided that the estimate remains available within the allowed visual-staleness period.
12. **Compute relative motion errors:** Calculate the rover-minus-drone horizontal position and velocity errors and determine the corresponding distance and relative speed.
13. **Update the flight-state machine:** Evaluate the current conditions and transition between SEARCH → TRACK → DESCEND → FINAL → CUTOFF. Descent requires sufficiently small position and relative-velocity errors for the required duration. Misalignment or prolonged visual loss causes the controller to reduce/stop descent or return to tracking.
14. **Generate the height reference:** Determine the desired altitude and reference vertical velocity according to the current flight phase and altitude-dependent descent schedule. Reduce or stop the descent when alignment or visual-tracking conditions become unsafe.

15. **Generate horizontal and vertical control:** Calculate the desired horizontal acceleration using rover acceleration feed-forward, proportional position correction, velocity correction, and wind compensation. Calculate the desired vertical acceleration using altitude error, vertical-velocity error, and vertical disturbance compensation.
16. **Calculate the desired physical force:** Combine the desired acceleration with gravity and multiply by the drone mass to obtain the required three-dimensional physical force.
17. **Determine the desired thrust direction:** Normalize the desired physical force to obtain the required body-up/thrust direction.
18. **Generate the desired attitude:** Construct the desired rotation matrix using the required thrust direction while preserving the current yaw heading. Calculate the attitude error and generate the required roll/pitch torque using rotational PD control.
19. **Calculate total thrust:** Determine the total thrust required to produce the desired physical force along the current body-up direction and apply the implemented thrust limits.
20. **Perform motor mixing:** Distribute the total thrust and roll/pitch torque among the four motors using the quadrotor force-allocation equations. Clamp each motor force to a minimum of zero.
21. **Return motor forces and update the next cycle:** Return $[f_1, f_2, f_3, f_4]$ to the simulator, store the applied thrust and previous state information required for the next wind-estimation and angular-velocity calculations, and begin the next 240 Hz control cycle.

17. Safety and Landing Constraints

The local scorer defines the external success/failure constraints used by the supplied test environment.

Condition	Implemented threshold	Interpretation
Ground collision	$z < 0.08$ m	Failure if the drone passes below this height.
Maximum safe descent velocity	-0.65 m/s	At touchdown height, more negative vertical velocity is a failure.
Maximum safe tilt	0.52 rad $\approx 30^\circ$	At touchdown, excessive roll/pitch tilt is a failure.
Rover surface	$z = 0.15$ m	Reference rover surface used by the scorer.
Horizontal touchdown region	error < 0.5 m	Scorer checks touchdown behavior within this radius.
Settled vertical velocity	$ v_z < 0.05$ m/s	Required for success after the settling condition.
Scorer time condition	simulation time > 2.0 s	Success check requires sufficient elapsed simulation time.

The controller contains additional internal landing logic. In FINAL, it recognizes a deck condition at $CUT_Z = 0.152$ m or at a slightly higher height when upward/near-zero vertical motion indicates contact. It then requires horizontal distance below 0.35 m before switching to CUTOFF.

18. Results and Observations

The implemented controller was evaluated in both the standard PyBullet arena and the wind-enabled arena. In both test cases, the drone successfully tracked the moving rover, synchronized its horizontal motion, performed a controlled descent, transitioned to the FINAL phase, detected touchdown, and commanded motor cut-off.

18.1. Standard Arena — No Wind

In the no-wind run, the initial horizontal tracking error was approximately 0.58 m . The error reduced below 0.1 m before descent, with relative velocity approximately 0.07 m/s at the TRACK-to-DESCEND transition.

The drone entered FINAL at approximately 0.41 m altitude and reached the rover deck at 0.151 m . Motor cut-off was successfully confirmed.

18.2. Wind-Enabled Arena

In the wind-enabled run, wind disturbance was detected at approximately 0.42 s . The initial horizontal error was approximately 0.55 m and decreased to 0.09 m , with relative velocity approximately 0.06 m/s by 2.52 s .

The drone entered FINAL at approximately 0.41 m altitude and reached the rover deck at 0.151 m . Motor cut-off was successfully confirmed.

18.3. Comparison of recorded runs

Parameter	No wind	Wind enabled
Landing time	7.32 s	7.64 s
Touchdown height	0.151 m	0.151 m
Touchdown vertical velocity	-0.18 m/s	-0.35 m/s
Horizontal touchdown error	0.008 m	0.011 m
Touchdown tilt	4.9°	3.4°
Motor cutoff	Confirmed	Confirmed
Result	Safe touchdown	Safe touchdown

18.4. Vision and Tracking Observations

The wind-enabled run switched from direct ArUco detection to model-based tracking at approximately 1.27 m , compared with approximately 0.81 m in the no-wind run. This allowed the controller to continue tracking when direct marker detection became less reliable at lower altitude.

18.5. Overall Observation

The recorded runs demonstrate successful rover tracking, controlled descent, touchdown, and motor cut-off in both tested conditions. These results represent **individual simulation runs and should not be interpreted as a statistically representative multi-trial evaluation.**

21. Limitations

The current implementation has several limitations:

- **Simulated perception:** Vision and sensing are evaluated using the simulated camera and environment rather than real hardware.
- **Marker dependence:** The system relies on the target visual marker; severe occlusion or complete loss of the marker can reduce tracking reliability.
- **Prediction uncertainty:** During extended visual loss, the Kalman filter relies on prediction, which can accumulate position error.
- **Simplified dynamics and disturbances:** The simulated actuator model and wind disturbance do not fully represent real quadrotor aerodynamics and actuator dynamics.

These limitations define the main gap between the current simulation-based implementation and a physical UAV deployment.

22. Future Improvements

The system can be extended in the following directions:

- **Real-world validation:** Deploy the perception and control pipeline on a physical UAV to evaluate camera, actuator, and environmental effects not represented in simulation.
- **Improved state estimation:** Combine additional sensing sources with the visual measurements to improve robustness during marker occlusion and longer visual-loss periods.
- **Advanced motion prediction:** Replace the constant-acceleration model with a model that better represents the rover's actual motion and maneuver changes.
- **More extensive evaluation:** Perform a larger number of trials across different rover trajectories, initial conditions, and disturbance levels to obtain statistically meaningful performance measures.

23. Conclusion

This project developed a simulation-based autonomous precision-landing system for a quadrotor landing on a moving rover in PyBullet. The system combines camera-based platform detection, ArUco and template-based tracking, Kalman filtering with Mahalanobis measurement validation, relative-motion estimation, staged flight-state control, and position-based horizontal and vertical control.

The desired accelerations are converted into attitude, thrust, and four motor forces, while external disturbance estimation provides compensation during wind-enabled operation. The controller was evaluated in both no-wind and wind-enabled simulations, with the recorded runs achieving rover tracking, controlled descent, touchdown, and motor cut-off.

The results demonstrate the feasibility of the proposed closed-loop approach within the supplied simulation environment. However, the reported numerical results are based on individual recorded runs rather than a statistically representative multi-trial evaluation.